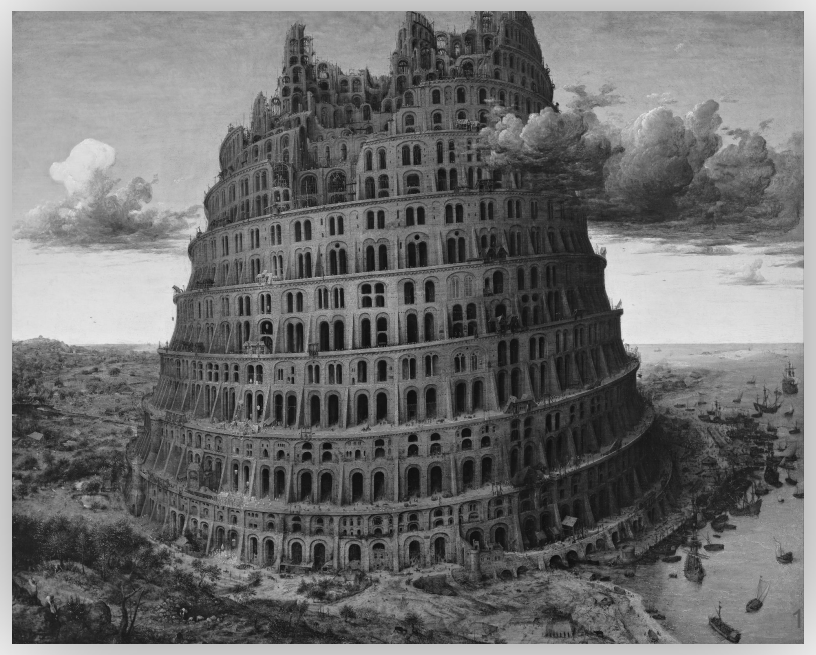
# Банда 4х против Гоферов

Стоит ли учить классические паттерны в 2026?



YADRO Go Course 26/1

# “Банда четырех” - все еще нужна гоферу в 2025?



# Зачем мне слушать эту презентацию?



Классические паттерны пригодятся, чтобы

- избежать "велосипедирования"
- говорить на одном языке с коллегами
- подбирать подходящие решения
- различать паттерны в чужом коде

Также обсудим

- что актуально, примеры в Go проектах
- современные аналоги
- что устарело и не заслуживает внимания *

# Что решают классические паттерны?



Код изменяется — новые требования и баги

Как решают:

- отделяют изменчивые части от неизменяемых
- используют интерфейсы, внедряют зависимости
- композиция — ключ к решению

# Объекты в Го - ООПГ



- Пакетная инкапсуляция
- Утиный полиморфизм
- Встраивание структур
- Композиция интерфейсов
- Функции - тоже объекты



“Скажем *НЕТ* наследованию имплементации”

# Брать нельзя выкинуть

Где поставим запятую?

# Классические паттерны

| | порождающие | структурные | поведение |
|---|---|---|---|
| классы | • фабричный метод | • адаптер | • интерпретатор<br>• шаблонный метод |
| объекты | • абстрактная фабрика<br>• одиночка<br>• прототип<br>• строитель | • адаптер<br>• декоратор<br>• заместитель<br>• компоновщик<br>• мост<br>• приспособленец<br>• фасад | • итератор<br>• команда<br>• наблюдатель<br>• посетитель<br>• посредник<br>• состояние<br>• стратегия<br>• хранитель<br>• цепочка обязанностей |

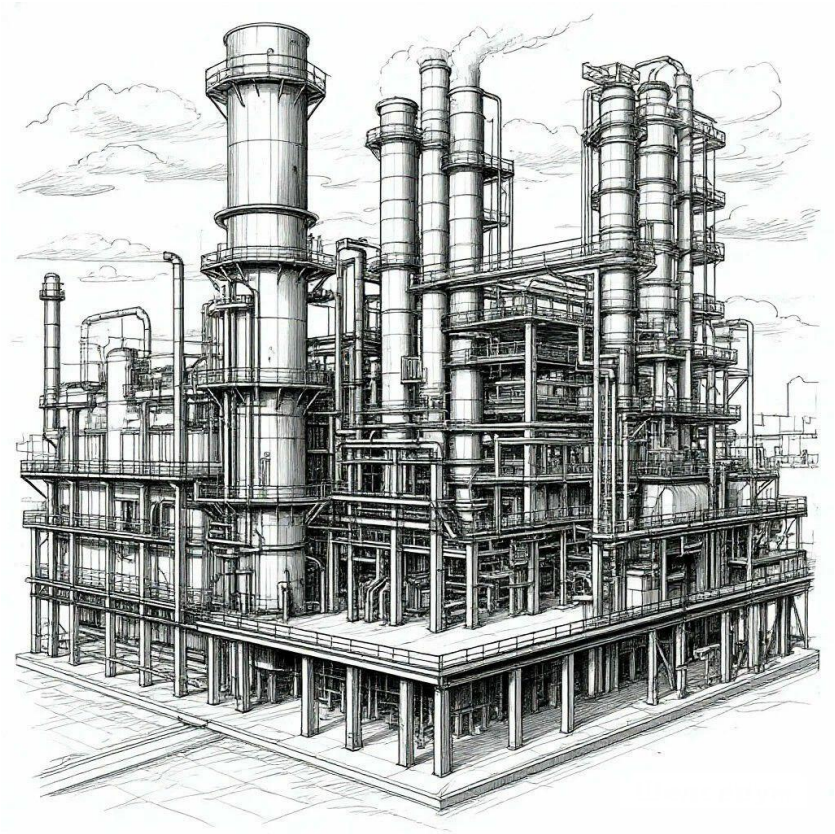
# Классические паттерны без наследования

| | порождающие | структурные | поведение |
|---|---|---|---|
| классы | • фабричный метод | • адаптер | • интерпретатор<br>• шаблонный метод |
| объекты | • абстрактная фабрика<br>• одиночка<br>• прототип<br>• строитель | • адаптер<br>• декоратор<br>• заместитель<br>• компоновщик<br>• мост<br>• приспособленец<br>• фасад | • итератор<br>• команда<br>• наблюдатель<br>• посетитель<br>• посредник<br>• состояние<br>• стратегия<br>• хранитель<br>• цепочка обязанностей |

# Порождающие паттерны



- Передают ответственность за создание объектов другим
- Делают систему независимой от способа создания, композиции и представления объектов




Фабрика!
Строитель!
Фабричный метод!
нам нужен общий язык!
Прототип!

# Простая фабрика



- Паттерн не из “банды 4х”
- Конструктор
- Скрывает внутренности объекта
- Может быть
  - функцией
  - методом
  - полем структуры (первоклассная функция)

# Простая фабрика



```
type Parser interface {
        Parse(io.Reader) []string
}
```

# Простая фабрика



```
type (
    LogParser struct{}
    ReportParser struct{}
    PlainTextParser struct{}
)

func (l LogParser) Parse(data io.Reader) []string {
    ...
}
```

# Простая фабрика



```
func New(filename string) Parser {
    // each ext has own parser
    switch filepath.Ext(filename) {
        case ".log": return LogParser{}
        case ".json": return ReportParser{}
    }
    return PlainTextParser{}
}
```



# Фабричный метод



- Фабрика спрятана в наследнике
- Знания о конкретных объектах откладываются на потом
- Не работает в Go ..

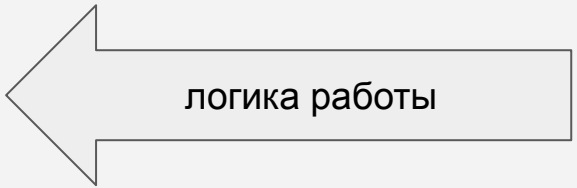
# Фабричный метод

```
type FileProcessor struct{}

func (fp FileProcessor) Process(files []string) {
    parser := fp.GetParser() // "запчасть" извне

    for _, f := range files {
        parser.Parse(filename)
    }
}

func (fp FileProcessor) GetParser() Parser {
    panic("not implemented")
}
```



# Фабричный метод

```
type LogProcessor struct {
   FileProcessor
}

func (fp LogProcessor) GetParser() LogParser {
   return LogParser{}
}
```

# Фабричный метод

```
func TestFactoryMethod(t *testing.T) {
    lp := factorymethod.LogProcessor{}
    lp.Process()
}

--- FAIL: TestMethod (0.00s)
panic: not implemented [recovered]
    panic: not implemented
```

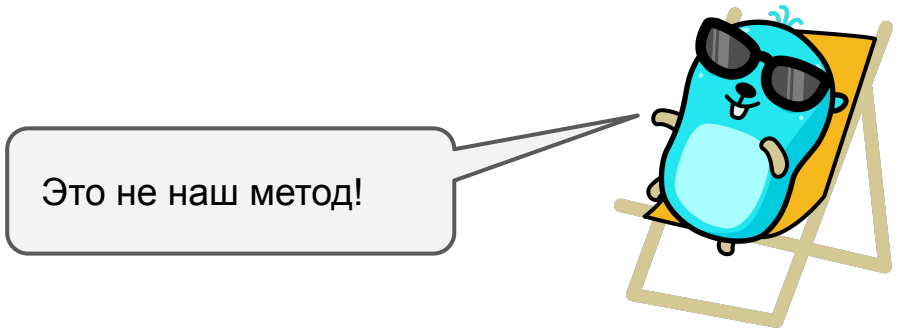
# Фабричный метод

👍 Плюсы: нет

⛔ Минусы: невозможно реализовать

🆚 Альтернативы: простая фабрика

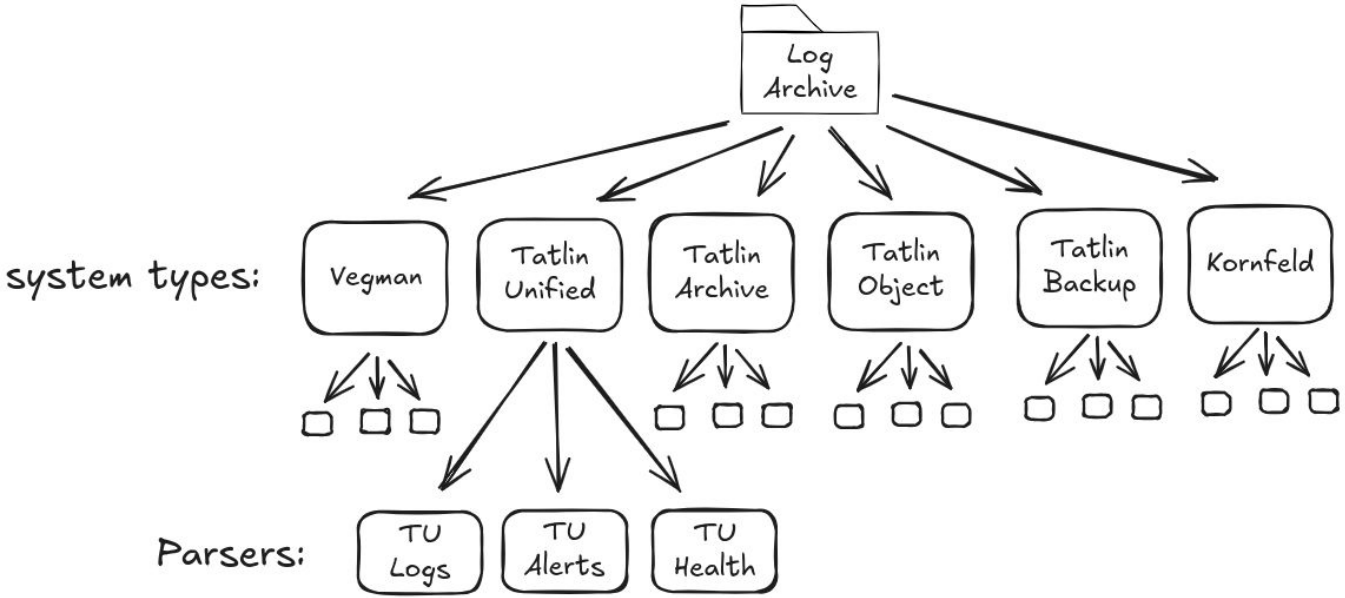
🏁 Вердикт: нет!



# Абстрактная фабрика



- Фабрика фабрик
- Создает согласованные семейства объектов
- Например, набор парсеров для разбора архива логов от нашей системы



# Без абстрактной фабрики



```
for _, f := range files {

    switch systemType {
        case Tatlin.Unified:
            switch filepath.Ext(f) {
            case ".log":
                TULogParser{}.Parse(f)
            case ".json":
                TUReportParser{}.Parse(f)
            }
        case Tatlin.Backup:
            // show must go on
            // and on
            // and
```

# Абстрактная фабрика - заготовка



```
type LogParser interface {
   Parse(filename string)
}

type ReportParser interface {
   Parse(filename string)
}

type Factory interface {
   NewLogParser() LogParser
   NewReportParser() ReportParser
}
```

# Абстрактная фабрика - имплементации



```
type TUFactory      struct{}
type TULogParser    struct{}
type TUReportParser struct{}

func (tlp TULogParser) Parse(filename string) {
   fmt.Println("parsing TU log", filename)
}

func (tuf TUFactory) NewLogParser() LogParser {
   return TULogParser{}
}
```

# Абстрактная фабрика



```
func factoryMaker(sysType string) Factory {
    switch sysType {
        case "TB":
            return TBFactory{}
        case "TU":
            return TUFactory{}
    }
    panic("unknown system type")
}
```

# Абстрактная фабрика



```
factory := factoryMaker(sysType)
logParser := factory.NewLogParser()
reportParser := factory.NewReportParser()

for _, f := range files {
    switch filepath.Ext(f) {
    case ".log":
        logParser.Parse(f)
    case ".json":
        reportParser.Parse(f)
    }
}
```

# Примеры абстрактной фабрики в Go?

В стандартной библиотеке нет именно абстрактной фабрики, но есть похожее (database/sql):

```
SQL Driver interface → Open() →
    Conn interface → Prepare() →
    Stmt interface → Query() →
    Rows interface → Next()
```

Конкретный драйвер имплементирует всю цепочку.



# Абстрактная фабрика

👍 Плюсы: согласованность, расширяемость, помогает при больших иерархиях объектов

⛔ Минусы: все еще куча кода, но уже по частям

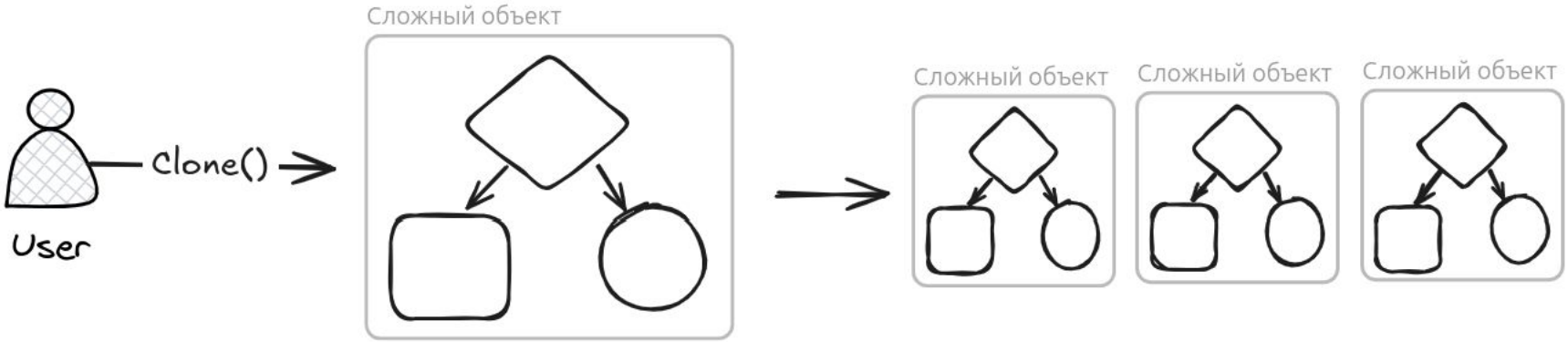
🆚 Альтернативы: simple factory, prototype, mega-switch

🏁 Вердикт: может быть!

# Прототип



- Экземпляр-прототип задает виды создаваемых объектов
- **Clone()** вместо сложного конструктора (много параметров, дерево, ..)
- Помогает избежать новые классы и иерархии
- Клиенту не нужно знать о конкретных классах прототипов - работает с **интерфейсами**



# Прототип - пример

Создавать профили пользователей на основе шаблонов:

- User
- Tester
- Admin



# Прототип - пример

```
type Profile interface {
    Clone() Profile
    Print() string
    SetName(name string)
    AddPermission(perm string)
}
```

# Прототип - пример

```
type user struct {
    name        string
    permissions []string
}

func NewUser() *user {
    return &user{
        name:        "unset",
        permissions: []string{"work", "party"},
    }
}
```

# Прототип - пример

```
func (u user) Clone() Profile {
   return &user{
       name:        u.name,
       permissions: slices.Clone(u.permissions),
   }
}

func (u *user) SetName(name string) {..}
func (u *user) AddPermission(perm string) {..}
func (u *user) Print() string {..}
```

# Прототип - пример

```
type tester struct {
   user
}

func NewTester() *tester {
   var t tester
   t.name = "unset"
   t.permissions = []string{"break things", "open bugs"}
   return &t
}
```

# Прототип - пример

```
func (t *tester) Clone() Profile { .. }
func (t *tester) Print() string { .. }
// остальное из embedded user
```

# Прототип - пример

```
type admin struct {
   user
}

func NewAdmin() *admin { .. }
func (a *admin) Clone() Profile { .. }
func (a *admin) Print() { .. }
```

# Прототип - пример

```
prototypes := make(map[string]Profile)

prototypes["normal"] = NewUser()
prototypes["testing"] = NewTester()
prototypes["superpower"] = NewAdmin()
```

# Прототип - пример

```
// create tester
user1 := prototypes["testing"].Clone()
user1.SetName("Olga")
user1.AddPermission("run nightly tests")
card1 := user1.Print()

// create admin
user2 := prototypes["superpower"].Clone()
user2.SetName("Kostya")
card2 := user2.Print()
```

# Пример из библиотеки Go, http.Request

```
request, err := http.NewRequest(
    http.MethodGet,
    "http://www.abc.com",
    nil,
)

request.Header.Add("Authorization", "token 123")
request.Header.Add("Accept", "application/json")
    ...
```

# Пример из библиотеки Go, http.Request

```
// почти полный клон
get := request.Clone(context.TODO())


// “тонкий” клон
post := request.WithContext(context.TODO())
post.Method = http.MethodPost
```

# Прототип

👍 Плюсы: скорость, простота

⛔ Минусы: нет

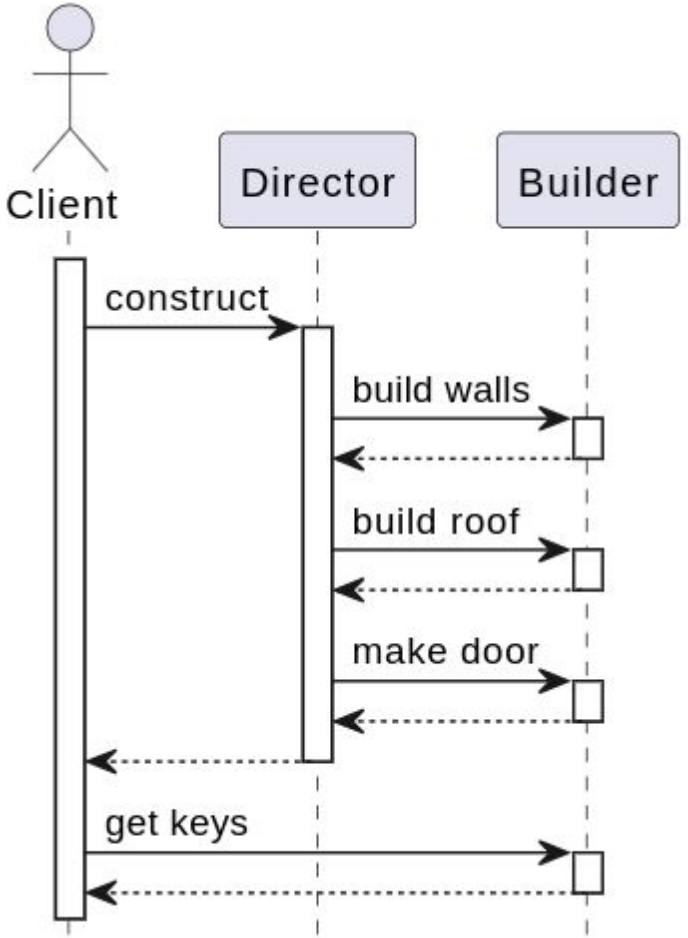
🆚 Альтернативы: simple factory, singleton, composite, flyweight

🏁 Вердикт: может пригодиться, если Вы - геймдев

# Строитель



- Процесс конструирования сложного объекта независимо от представления
- Один процесс - много разных представлений:
  - изба
  - барнхаус
  - фахверк



# Строитель для таблиц - абстракция



```
type Builder interface {
    AddRow([]string)
    AddHeader([]string)
    AddFooter([]string)
    Build() []byte
}

type ReportManager struct {
    builder Builder
}
```

# Директор строительства

```
func (m Manager) NewReport(stats [][2]string) []byte {

    m.builder.AddHeader([]string{"name", "status"})

    for _, row := range stats {
        m.builder.AddRow(row[:])
    }
    m.builder.AddFooter([]string{"---", "---"})

    return m.builder.Build()
}
```

# Строитель для таблиц CSV

```
type CSV struct {
   buffer bytes.Buffer
}

func (c *CSV) AddHeader(header []string) {
   fmt.Fprintln(&c.buffer, strings.Join(header, ","))
}
 ...

func (c *CSV) Build() []byte {
   return c.buffer.Bytes()
}
```

# Строитель для таблиц

```
var diskStats = [][2]string{
   {"disk1", "ok"},
   {"disk2", "failed"},
   {"disk3", "ok"},
}

m := report.NewManager(&builders.CSV{})
csv := m.NewReport(diskStats)

// name,status
// disk1,ok
// disk2,failed

//  ...
```

# Go way - текучий интерфейс



```
type Server struct {
   address string
   port    int
   secure  bool
}

func Default() *Server {
   return &Server{ port: 8080 }
}
```

# Go way - текучий интерфейс



```
func (s *Server) Address(address string) *Server {
   s.address = address
   return s
}
...

func main() {
    s := fluent.Default().Address("0.0.0.0").Port(80)
    server.Run()
```

# Command Query Separation (CQS)



Принцип проектирования программного кода, требует четкого разделения двух типов операций:

1. Команды (Commands) — операции, изменяющие состояние объекта:
   - создают, обновляют или удаляют данные;
   - выполняют действия с побочными эффектами;
   - не должны возвращать значимые данные (только статус выполнения).

   Пример: func save(user User) error { .. }
2. Запросы (Queries) — операции, возвращающие информацию:
   - извлекают данные без изменения состояния;
   - не имеют побочных эффектов;
   - могут вызываться многократно без риска ошибок.

   Пример: func isAdmin(user User) bool

# Command Query Separation (CQS)



Ключевые правила

- Команда не возвращает данных (кроме статуса выполнения).
- Запрос не изменяет состояние системы.
- Формулировка-мантра: «Задавая вопрос, не изменяй ответ».

Почему это важно?

- Читаемость кода: ясно, какая функция что делает.
- Тестируемость: команды и запросы тестируются отдельно.
- Предсказуемость: запросы можно вызывать сколько угодно раз без побочных эффектов.
- Сопровождаемость: легче находить ошибки в разделенном коде.

# Текучий интерфейс - CQS



```
func (s Server) WithAddress(address string) Server {
    s2 := s1
    s2.address = address
   return s2
}
 ...
```

# Текучий интерфейс - CQS



```
base := Default().WithAddress("0.0.0.0").WithSecure()

server1 := base.WithPort(7777)
server2 := base.WithPort(8888)

server1.Run()
server2.Run()
```

# Go way - просто конструктор



```
package server

// port    - mandatory
// address - optional?, default 0.0.0.0
// secure  - optional?, default false
func New(
    port int, address string, secure bool, ...
) *Server {
    // build new Server here
}
```

# Go way - функциональные опции



```
type Config struct {
   port    int    // required
   address string // optional
   secure  bool   // optional
}

type Server struct {
    config Config
}

type Option func(Config) Config
```

# Функциональные опции



```
func WithSecure() Option {
    return func(config Config) Config {
        config.secure = true
        return config
    }
}

func WithAddress(address string) Option {
    return func(config Config) Config {
        config.address = address
        return config
    }
}
```

# Функциональные опции



```
package server

func New(port int, opts ...Option) *Server {
    config := Config{port: port}
    for _, o := range opts {
        config = o(config)
    }
    // do something smart here
    return &Server{config: config}
}
    // ..где-то в main
    s := New(8080, WithAddress("localhost"))
```

# Дисфункциональные опции



```
package config

// mandatory parameters
func New(port int) *Config {
   return &Config{port: port}
}
```

# Дисфункциональные опции



```
func (c *Config) SetSecure() *Config {
   c.secure = true
   return c
}

func (c *Config) SetAddress(addr string) *Config {
   c.address = addr
   return c
}
   ...
s := server.New(config.New(80).SetAddress("www"))
```

# Builder vs FunOpts vs DysOpts



| | Builder | FunOpts | DysOpts |
|---|---|---|---|
|  | • гибкость<br>• абстракция, SRP<br>• понятность<br>• Director - алгоритм | • добавление опций<br>• именованные аргументы в любом порядке | • легкость применения<br>• понятность |
|  | • много кода | • проверка ошибок<br>• boilerplate код<br>• непонятность<br>• неизвестность | • проверка ошибок<br>• non-CQS<br>• добавление опций |

# Строитель

👍 Плюсы: гибкость, инкапсуляция сложности, SRP, разные строители для тонкой настройки, разные директоры для разных алгоритмов

⛔ Минусы: много кода

🆚 Альтернативы: просто фабрика, funOpts, dysOpts

🏁 Вердикт: скорее да, чем нет, или?

# Singleton

- Гарантирует, что в приложении будет единственный экземпляр некоторого класса
- Предоставляет глобальную точку доступа к этому экземпляру

# Ленивые и не очень “одиночки”

```
type singlelogger struct {
   l *slog.Logger
}

func (l *singlelogger) Log(x string) {
   l.l.Debug(x)
}

var logger *singlelogger
```

# init-"одиночки”

```
func init() {
   logger = &singlelogger{
       l: slog.New( .. ),
   }
}

func New() *singlelogger { return logger }
```

# Ленивые "одиночки”

```
var once sync.Once

func New() *singlelogger {
    once.Do(func() {
        logger = &singlelogger{
            l: slog.New( .. ),
        }
    })
    return logger
}
```

# Singleton?

*“Гарантирует, что в приложении будет единственный экземпляр некоторого класса”*

# Пакетные "одиночки” - настоящий singleton

```
package log

// .. global private single logger + init()

func Log(x string) {
   l.l.Debug(x)
}
```

---

```
package main
..
   log.Log("hello world!")
```

# Примеры singleton в Go

В стандартной библиотеке нет «чистого» Singleton.

Go избегает жестких шаблонов в пользу:

- Простых глобальных переменных
- Пакетного уровня инкапсуляции (экспорт одной переменной/функции)
- Явных механизмов синхронизации (sync.Once, sync.Mutex) для контроля инициализации

# Примеры singleton в Go

Экземпляр может быть не единственным, но будет одним, если захотим - возврат одной и той же структуры. Примеры:

- default http Client / Mux
- slog, log - единственные дефолтные
- SQL драйверы
- конфиги, пулы соединений, кешы

http default client

# Одиночка

👍 Плюсы: простота, экономия памяти

⛔ Минусы: глобальное состояние, гонки

🆚 Альтернативы: flyweight

🏁 Вердикт: да!

# Структурные паттерны



Компоновка объектов для получения новой функциональности

- удобство
- эффективность
- расширяемость

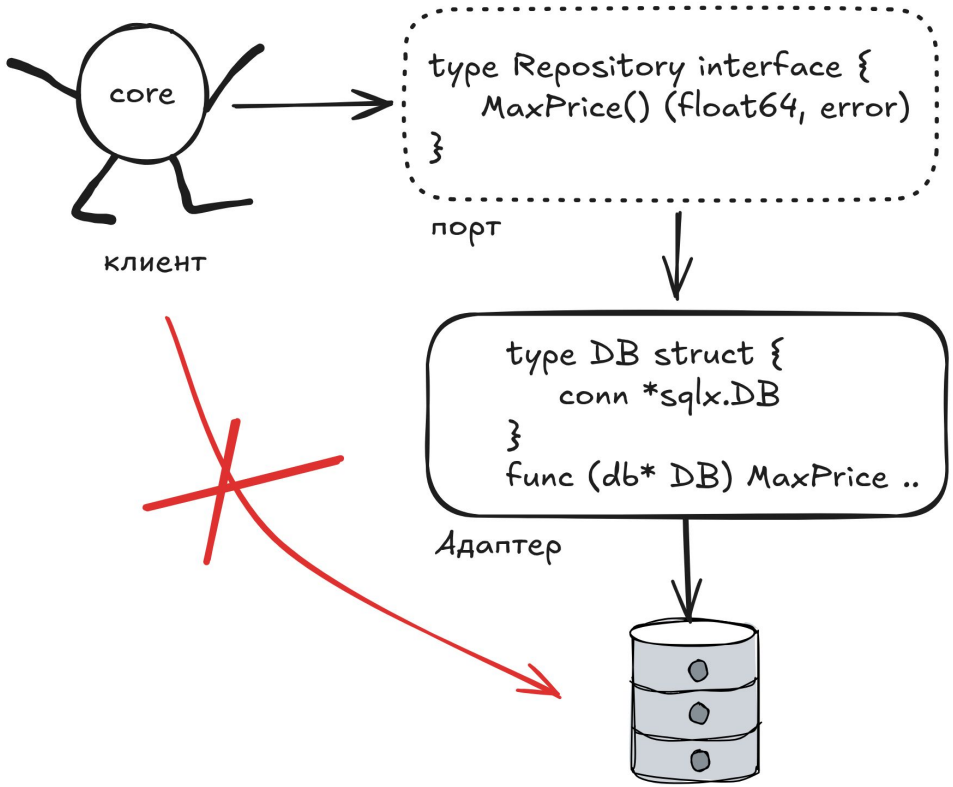


# Адаптер



Помогает адаптироваться одному объекту под другой

- объект-переводчик
- устранение жесткой связи
- основа ГА, ЧА & друзей



# Адаптер - классика

```
// Adapter
type DB struct {
   conn *sqlx.DB // Adaptee
}

// adapt port for core
func (db DB) MaxPrice() (float64, error) {
   var price float64
   err := db.conn.Get(
       &price, "SELECT max(price) FROM cars")
   return price, err
}
```

# Адаптер - embed

```
// Adapter
type DB struct {
   *sqlx.DB // Adaptee
}

// adapt port for core
func (db DB) MaxPrice() (float64, error) {
   var price float64
   err := db.Get(&price, "SELECT max(price) FROM cars")
   return price, err
}
```

# Адаптер - type



```
// Adapter is of type Adaptee
type DB sqlx.DB

func (db *DB) MaxPrice() (float64, error) {
    var price float64
    err := (*sqlx.DB)(db).Get(
        &price, "SELECT max(price) FROM cars")
    return price, err
}
```

# Адаптер - type без методов?



```
type X int
func (x X) Fx() int {
    return int(x)
}

type Y X
func (y Y) Fy() int {
    // Не работает - тип не наследует методы
    return y.Fx()
}
```

# Адаптер - type со звездочкой



```
// Не работает
//     - тип ресивера не может быть * или interface

type X *int
func (x X) Fx() int {
    return *((*int)(x))
}
```

# Адаптер - type func



```
// Отлично работает!
type HandlerFunc func(ResponseWriter, *Request)

// ServeHTTP calls handler(w, r).
func (handler HandlerFunc) ServeHTTP(
          w ResponseWriter, r *Request,
) {
    handler(w, r)
}
```

# Адаптер - closure



```
func NewMaxPrice(conn *sqlx.DB) func() (float64, error) {
   return func() (float64, error) {
       var price float64
       err := conn.Get(&price,
          "SELECT max(price) FROM cars")
       return price, err
   }
}
    ...
   conn := sqlx.MustConnect("postgres", dsn)
   mx := db.NewMaxPrice(conn)
   price, err := mx()
```

# Адаптеры



| | Struct | Embed | Type | Closure / Fn |
|---|---|---|---|---|
|  | просто, понятно, масштабируется | меньше кода (просто добавляем методы), “двусторонние адаптеры” | минимум кода | быстро, просто |
|  | больше кода, чем хочется | открытость | приведение типов (кроме для функций) | не подходит для сложных интерфейсов и большой вложенности |

# Адаптер

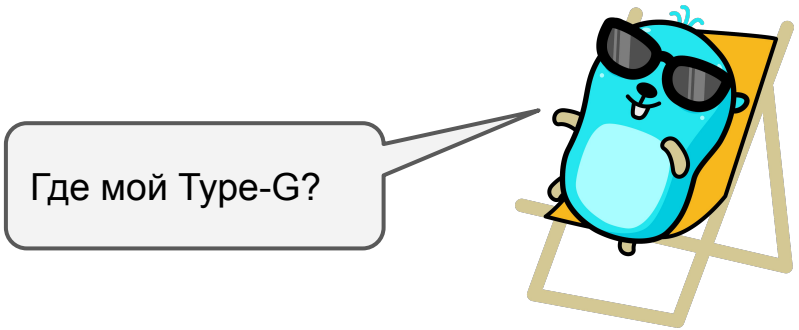


👍 Плюсы: простота

⛔ Минусы: недостаточная гибкость (post factum)

🆚 Альтернативы: bridge

🏁 Вердикт: да!



# Мост

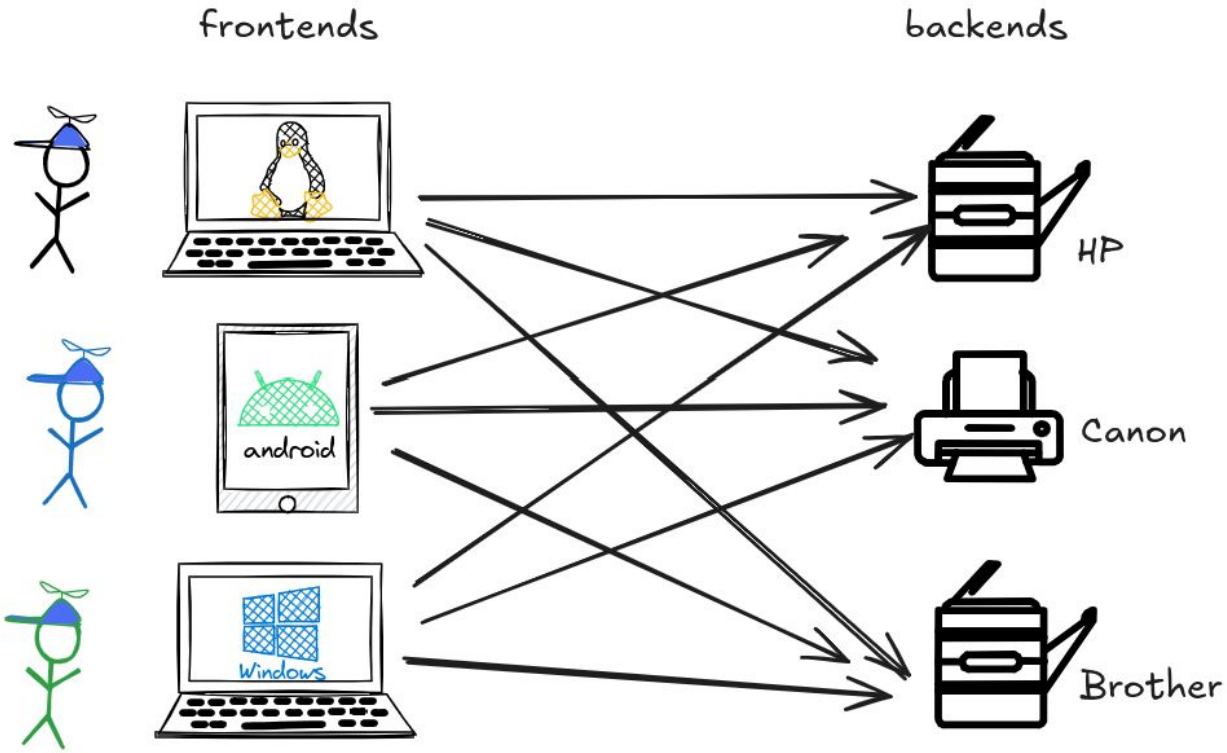


- Отделяет абстракцию от реализации
  - фронтенды от бэкендов
  - драйверы от сервисов
- Абстракцию и реализацию можно изменять независимо
- Иерархию N x M заменяет на N + M
- По сравнению с адаптерами проектируются **заранее**



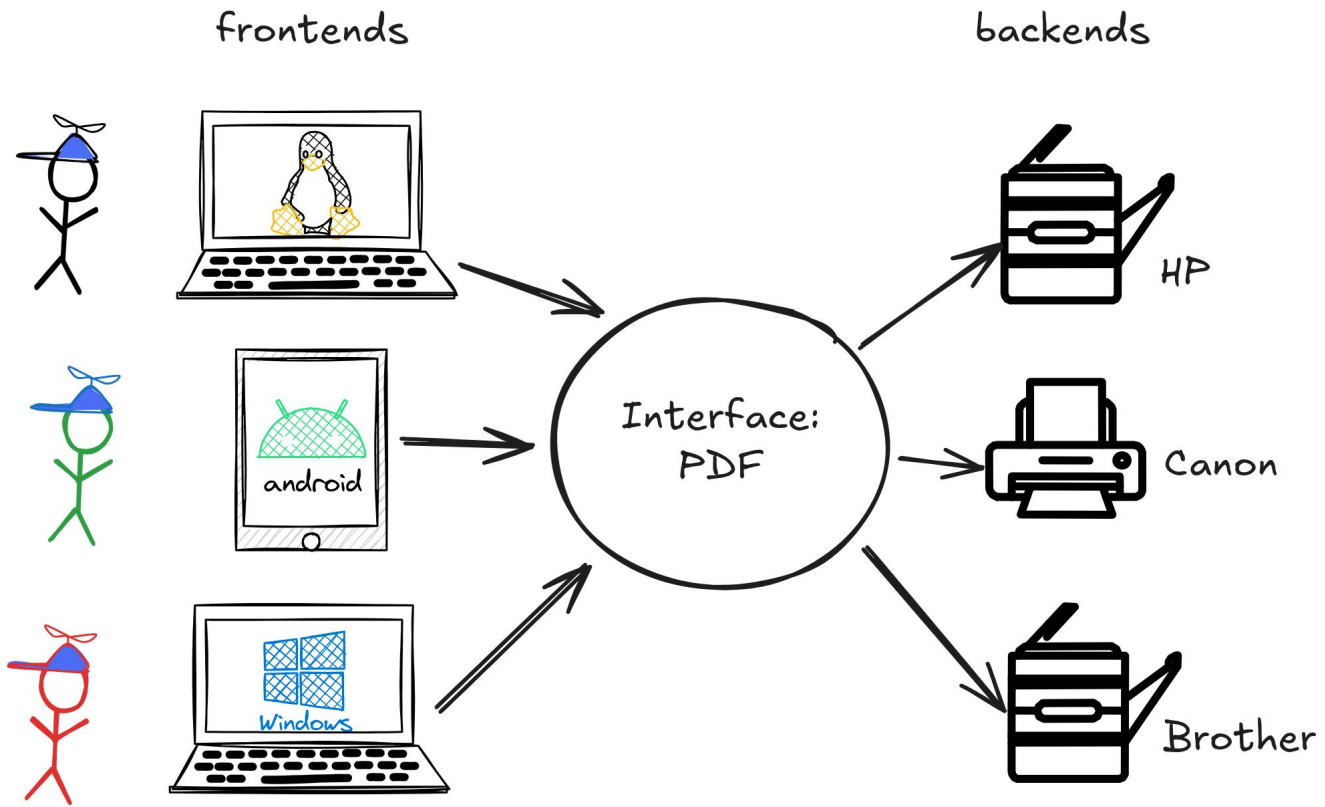
# Pre-Мост





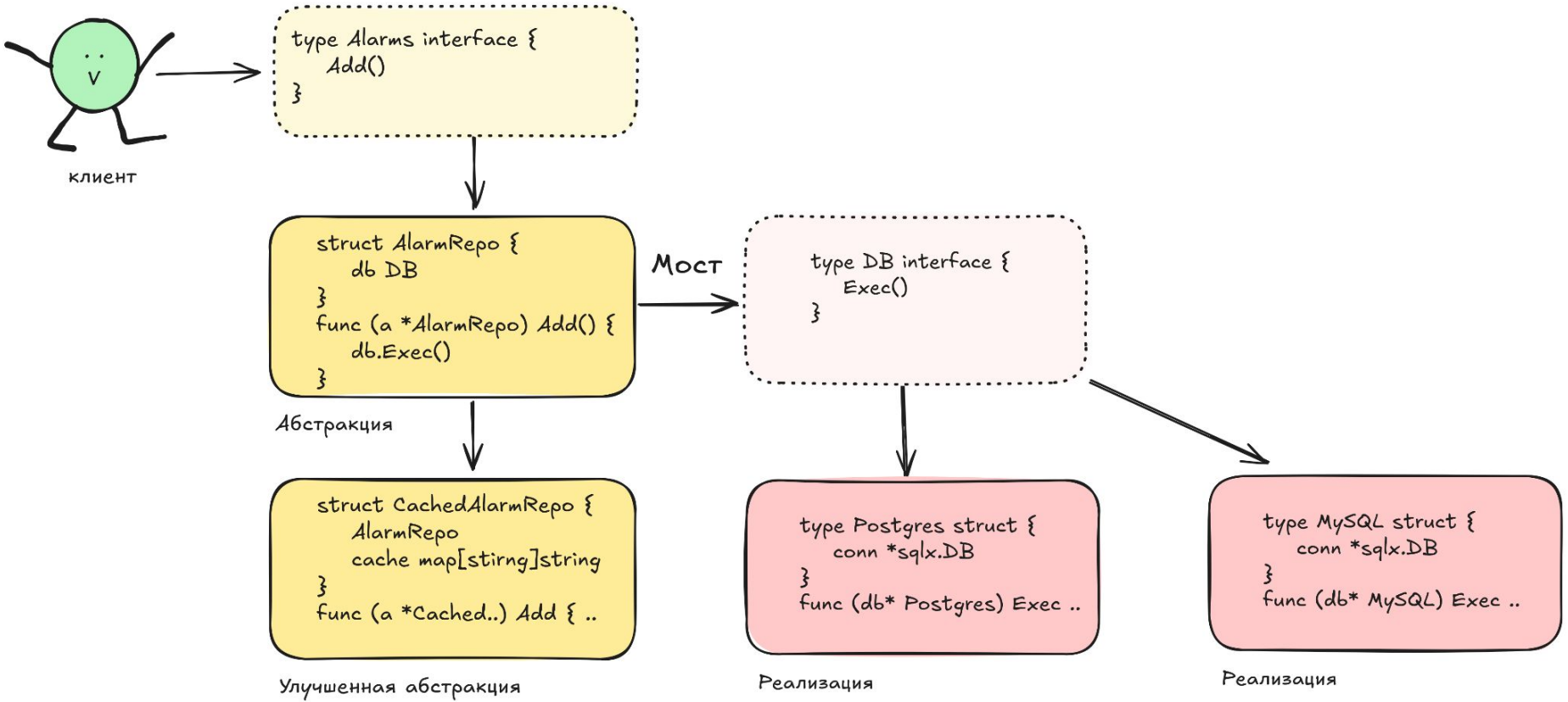
# Мост - операционки vs принтеры





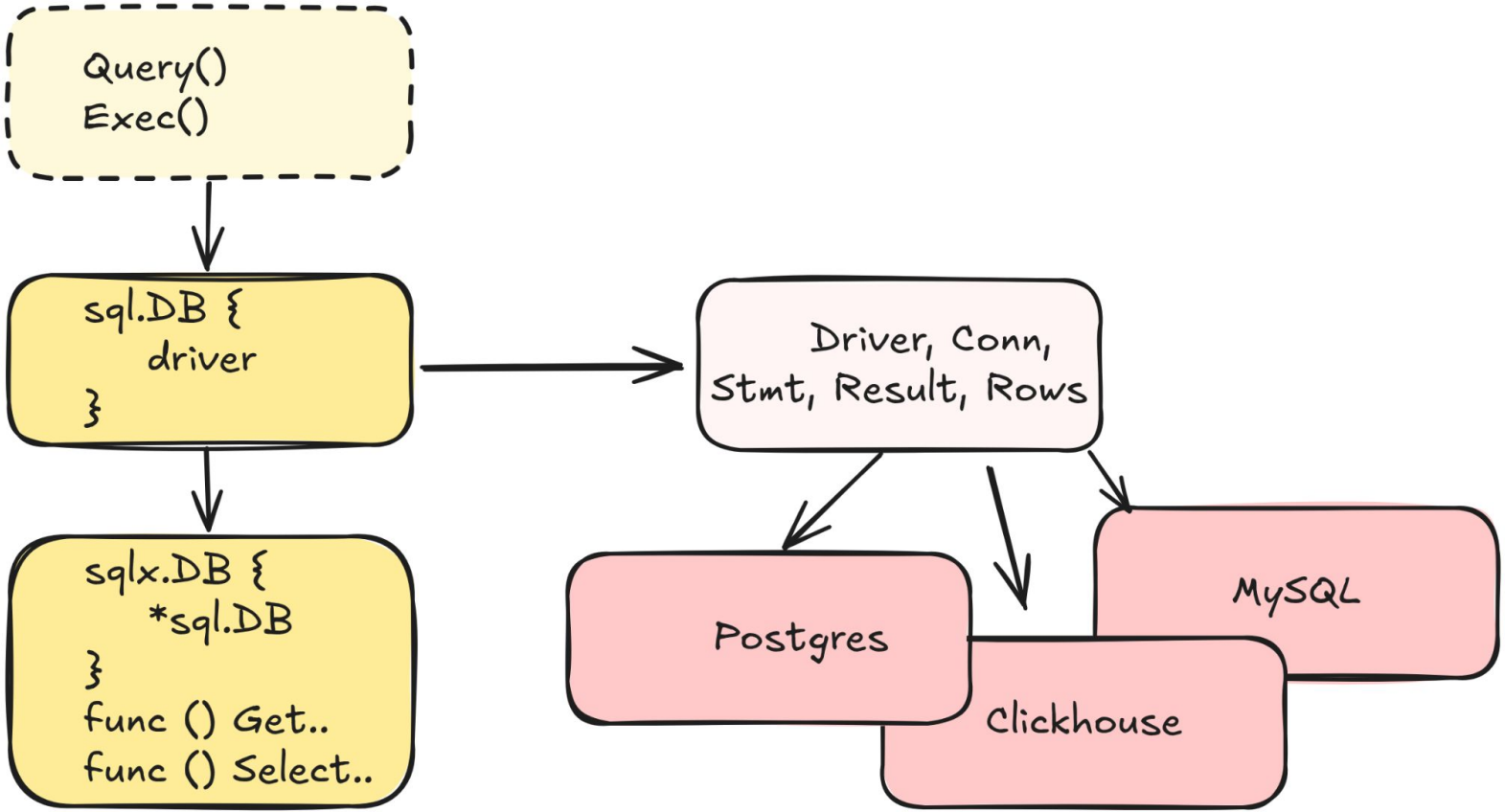
# Мост




клиент
type Alarms interface {
Add()
}
struct AlarmRepo {
db DB
}
func (a *AlarmRepo) Add() {
db.Exec()
}
Абстракция
Мост
type DB interface {
Exec()
}
struct CachedAlarmRepo {
AlarmRepo
cache map[stirng]string
}
func (a *Cached..) Add { ..
Улучшенная абстракция
type Postgres struct {
conn *sqlx.DB
}
func (db* Postgres) Exec ..
Реализация
type MySQL struct {
conn *sqlx.DB
}
func (db* MySQL) Exec ..
Реализация

# Пример в стандартной библиотеке





# Мост



👍 Плюсы: гибкость, минимум кода при большом разнообразии решений

⛔ Минусы: скорость*, сложность*, необходимость*

🆚 Альтернативы: adapter

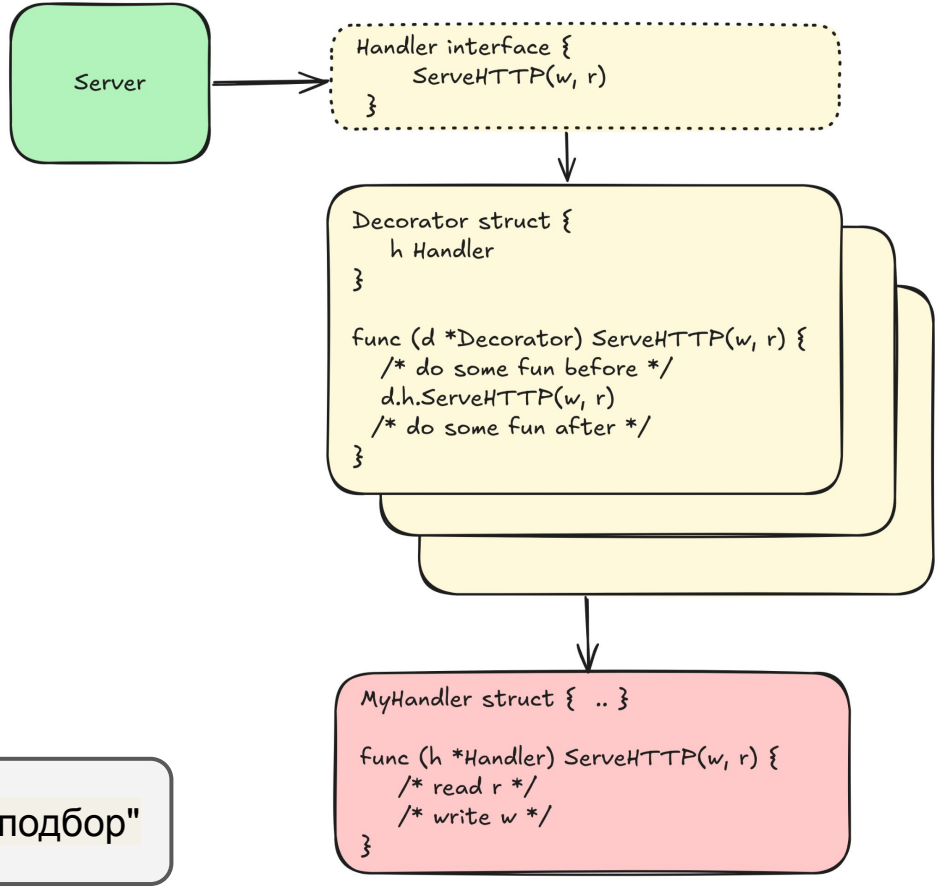
🏁 Вердикт: скорее нет, чем да

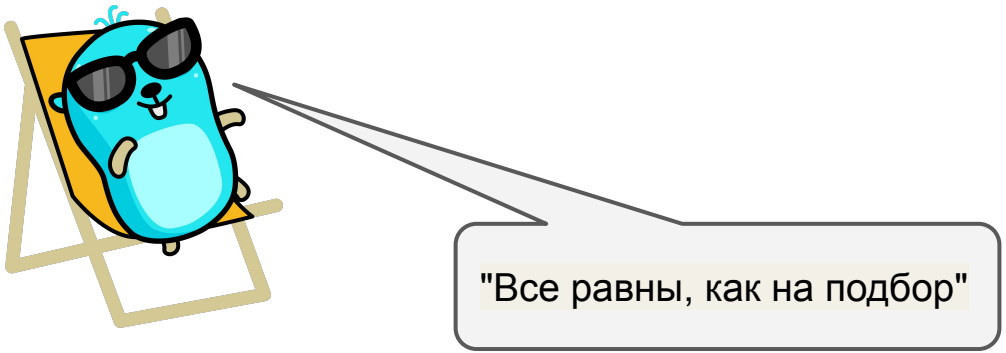
# Декоратор



Динамическое добавление объекту обязанностей

- Единственная ответственность
- Вложенность



# Декоратор - классика



```
type JSONHandler struct { /* more stuff here */ }

func (j JSONHandler) ServeHTTP(
    w http.ResponseWriter, r *http.Request,
) {

    var a map[string]any
    err := json.NewDecoder(r.Body).Decode(&a)
     ...
}
```

# Декоратор - классика



```
func main() {
   http.Handle("POST /json", JSONHandler{})
   http.ListenAndServe(":8888", nil)
}
```

# Декоратор - классика



```
type GzipDecorator struct {
   next http.Handler
}

func main() {
   http.Handle(
       "POST /json",
       GzipDecorator{ next: JSONHandler{} },
    )
   http.ListenAndServe(":8888", nil)
}
```

# Декоратор - классика



```
func (g GzipDecorator) ServeHTTP(w, r) {
   if r.Header.Get("Content-Encoding") == "gzip" {
       reader, err := gzip.NewReader(r.Body)
       if err ≠ nil {
           http.Error(w, ..)
           return
       }
       defer reader.Close()
       r.Body = io.NopCloser(reader)
       r.Header.Del("Content-Encoding")
   }
   g.next.ServeHTTP(w, r)
}
```

# Декоратор - замыкание



```
func Hello(w http.ResponseWriter, r *http.Request) {
   fmt.Fprintln(w, "hello there!")
}

func main() {
   http.HandleFunc("GET /hello", Hello)
   http.ListenAndServe(":8888", nil)
}
```

# Декоратор - замыкание



```
func WithLogger(
        l *slog.Logger, hf http.HandlerFunc,
) http.HandlerFunc {

    return func(w http.ResponseWriter, r *http.Request) {
        start := time.Now()
        hf(w, r)
        l.Info(
            "request",
            "url", r.URL, "duration", time.Since(start),
        )
    }
}
```

# Примеры из стандартной библиотеки



- bufio.Reader, gzip.Reader
- http.RoundTripper (на клиента - log, retry)
- iotest: OneByteReader, ErrReader, TimeoutReader, TruncateWriter ..

# Декоратор

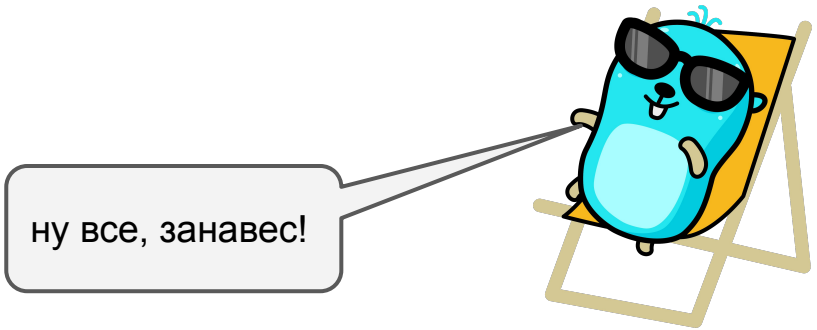


👍 Плюсы: гибкость, SRP

⛔ Минусы: много мелких объектов, сложно читать, тестировать, отлаживать

🆚 Альтернативы: стратегия, конвейер, фасад

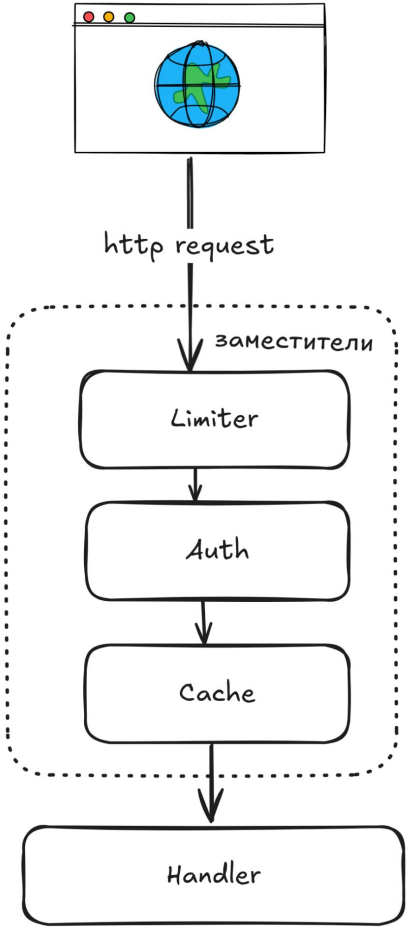
🏁 Вердикт: да!



# Заместитель

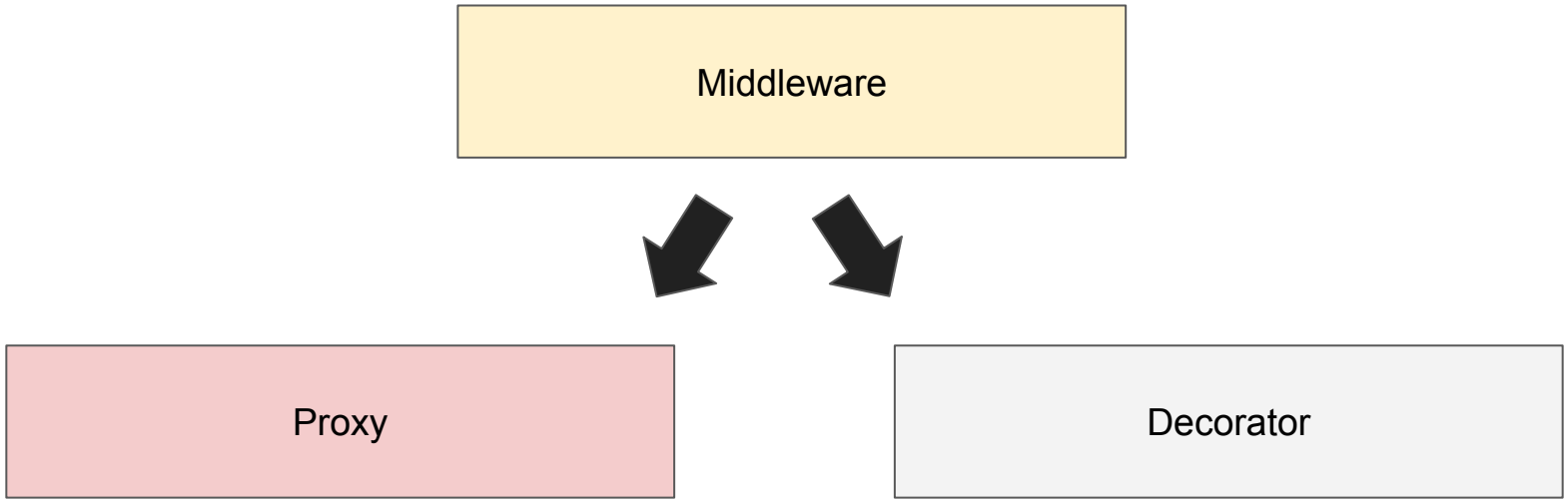


- Управляет доступом к объекту
  - удаленный
  - виртуальный
  - защищающий
  - ..
- Может управлять жизненным циклом замещаемого
  - В отличие от декоратора не добавляет функциональность, а управляет объектом



# Middleware / Interceptors






auth, rate, cache, validate

metrics, logging, timeout

# Заместитель - контроль доступа



```
func WithLimiter(
    concurrency int, hf http.HandlerFunc,
) http.HandlerFunc {
    sema := make(chan struct{}, concurrency)
    return func(w http.ResponseWriter, r *http.Request) {
        sema ← struct {}{}
        hf(w, r)
        ← sema
    }
}
```

..

```
http.HandleFunc("GET /hello", WithLimiter(100, Hello))
http.ListenAndServe(":8888", nil)
```

# Заместитель

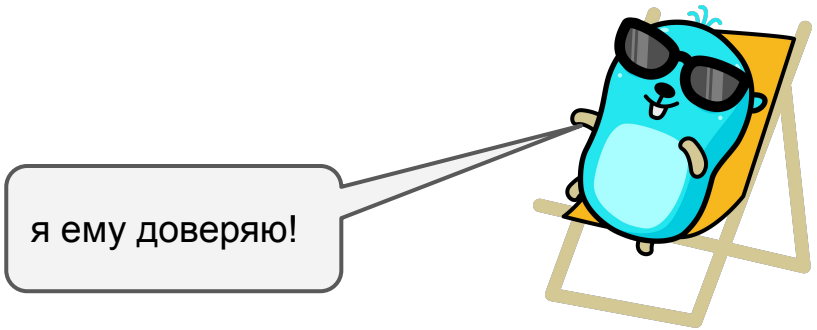


👍 Плюсы: гибкость, SRP

⛔ Минусы: много мелких объектов, сложно читать, тестировать, отлаживать

🆚 Альтернативы: фасад

🏁 Вердикт: да!

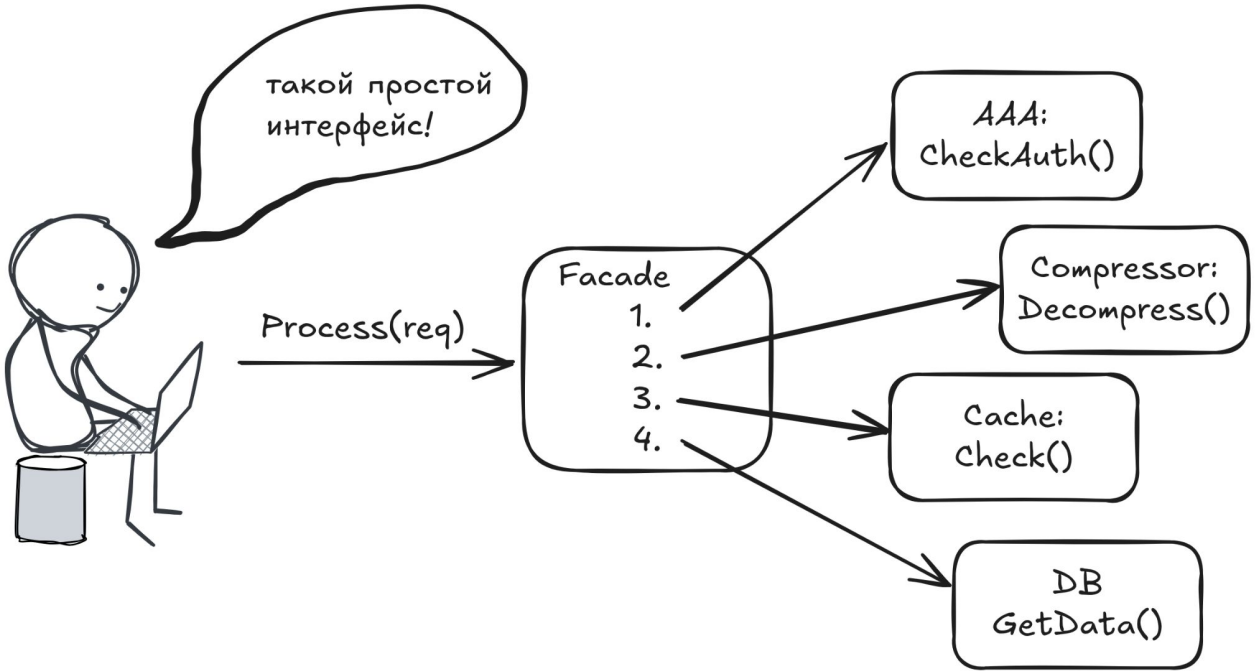


# Фасад



Простой интерфейс к сложной системе

- управляет подсистемами
- слабая связность
- абстракция



# Примеры из стандартной библиотеки



- sqlx для database/sql
  - Get
  - Select
  - Connect
- "одиночки" из http, rand, slog
  - многие фасады делаются как "одиночки"

# Фасад



👍 Плюсы: простота, удобство

⛔ Минусы: риск перерасти в “God object”, масштабирование и настройка

🆚 Альтернативы: много фасадов, скрытие фасада за интерфейсом

🏁 Вердикт: да!

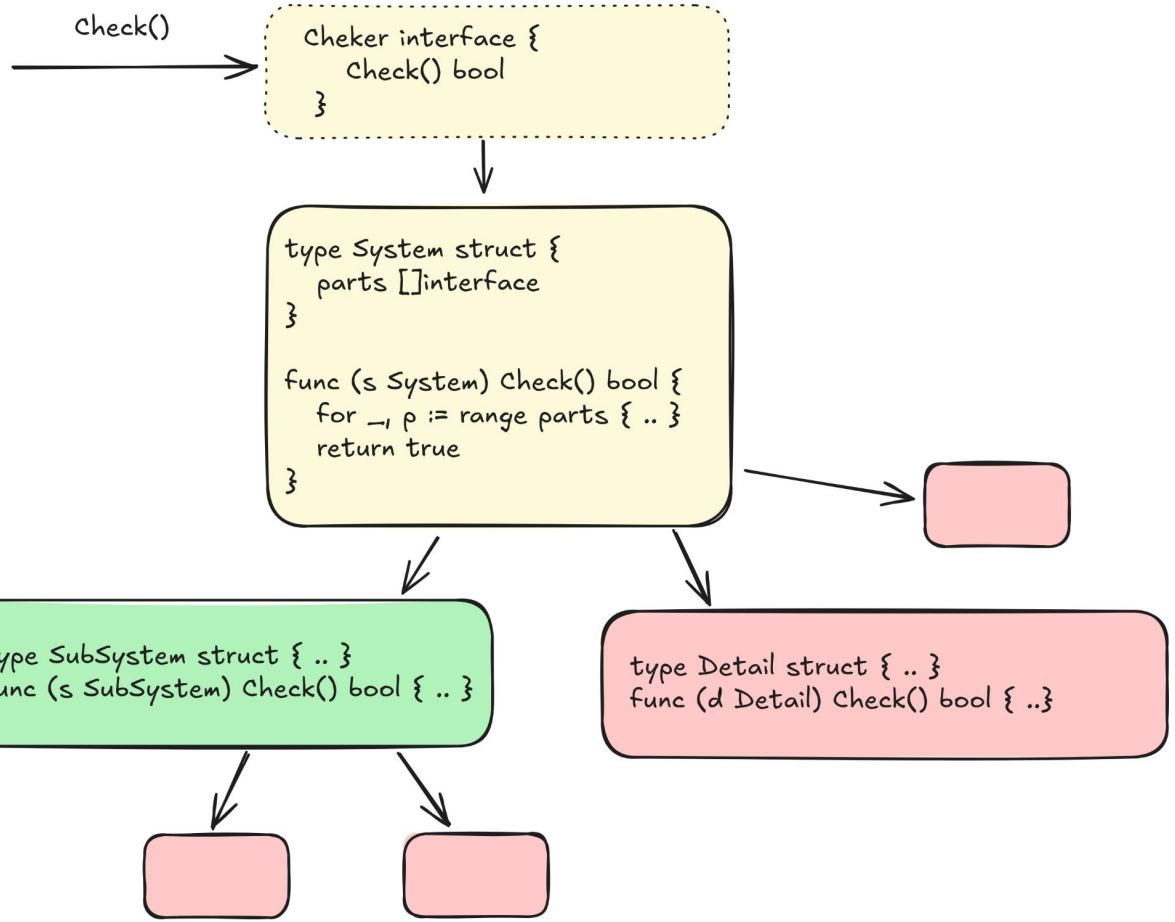


# Компоновщик



Работаем с древовидной структурой как с простым объектом

- гибкость
- простота
- расширяемость



# Компоновщик

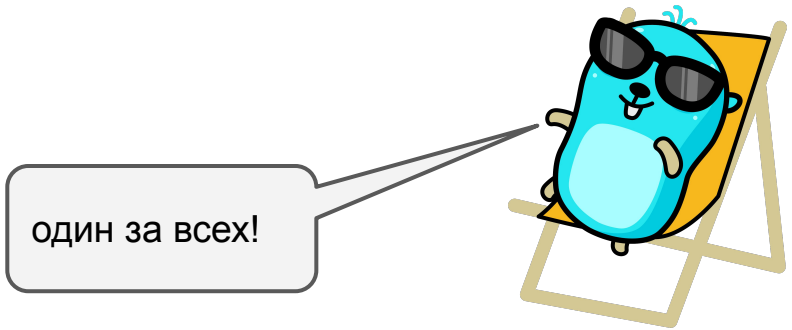


👍 Плюсы: гибкость, расширяемость, SRP

⛔ Минусы: множество мелких объектов, применимость

🆚 Альтернативы: простое дерево, список функций

🏁 Вердикт: скорее да, чем нет



# Приспособленец

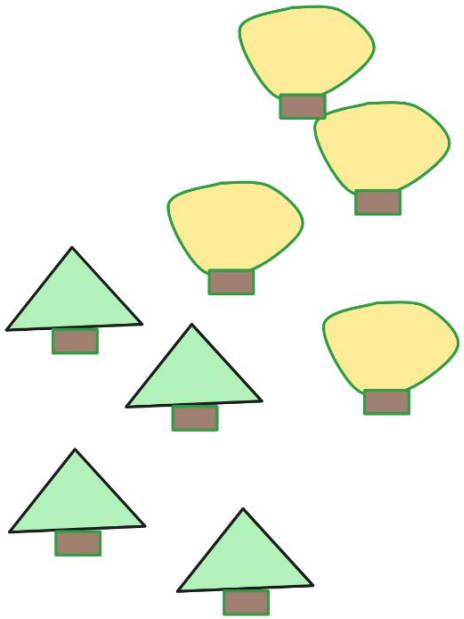


Экономия памяти за счет разделения объектов

- *очень много* объектов с повторяющейся частью
- внутреннее неизменное состояние - flyweight
- внешнее изменчивое состояние - context

# Приспособленец





```
Tree struct {
   fw *Fw
   x, y int
}
func (t Tree) Draw(x, y) {
   t.fw.Draw(x, y)
}
```

m — n

```
Fw struct {
   color string
   type string
}

func (f Fw) Draw(x, y) { ..}
```

```
TreeFactory stuct {
   fwRepo map[string]*Fw
}

func CreateTree(x, y, type, color) Tree {.. }
```

```
Fw:
color = 'yellow'
type = 'apple'
```

```
Fw:
color = 'green'
type = 'pine'
```

# Приспособленец

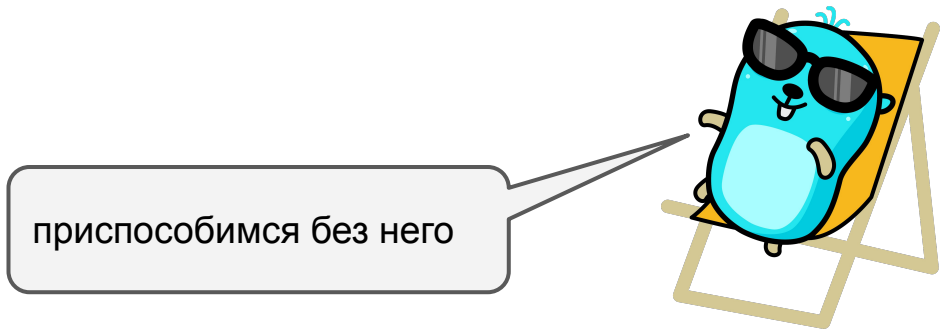


👍 Плюсы: экономия памяти, иногда - скорость

⛔ Минусы: много классов, сложно понять, поиск внутреннего контекста, можно ничего не сэкономить

🆚 Альтернативы: cache, singleton

🏁 Вердикт: скорее нет



# Паттерны поведения



Алгоритмы взаимодействия объектов

- инкапсуляция поведения
- уменьшение связности
- решение сложных задач
- эффективность

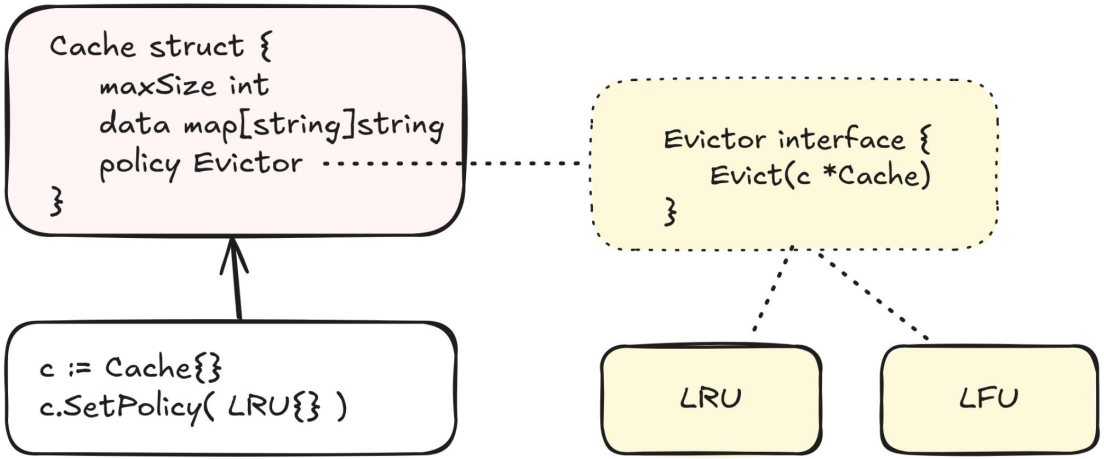


# Стратегия



Определяет семейство взаимозаменяемых алгоритмов

- настройка поведения
- меньше операций ветвления
- “горячая” замена
- отделение объекта от алгоритма



# Стратегия



```
names := []string{"Bob", "ALICE", "Gopher"}

strategy := func(a, b string) int {
    return strings.Compare(
        strings.ToLower(a), strings.ToLower(b),
    )
}

slices.SortFunc(names, strategy)
```

# Стратегия



👍 Плюсы: гибкость, простота, меньше if / switch

⛔ Минусы: много классов, затраты на передачу контекста, клиенты должны быть умнее

🆚 Альтернативы: если считать стратегией функцию - нет

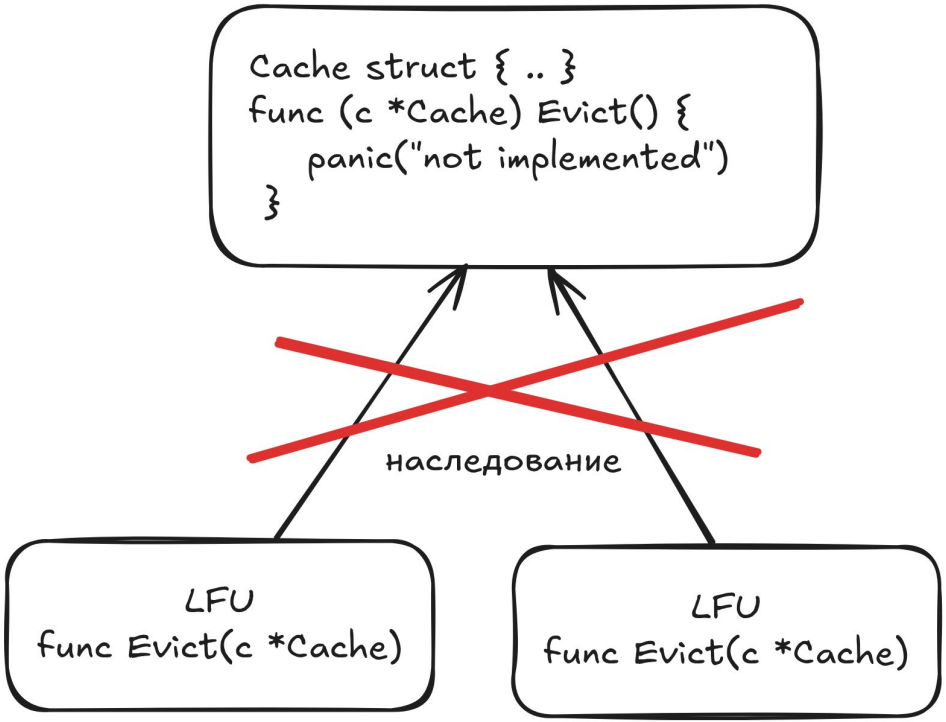
🏁 Вердикт: да!



# Шаблонный метод



- Алгоритм (или его часть) спрятан в **наследнике** класса
- Не работает в Го



# Шаблонный метод

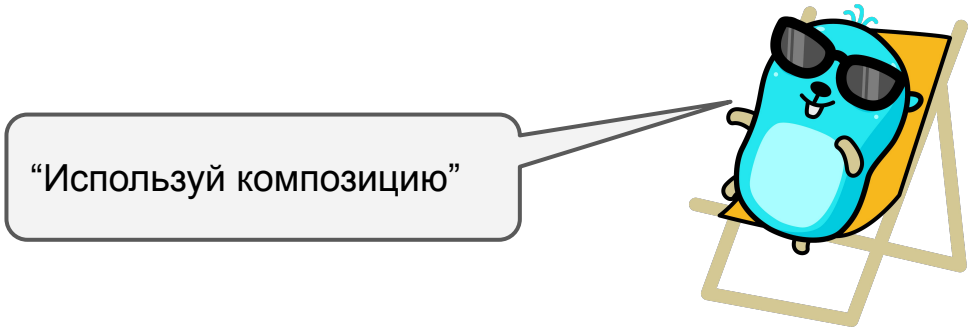


👍 Плюсы: нет

⛔ Минусы: невозможно реализовать

🆚 Альтернативы: стратегия

🏁 Вердикт: нет!

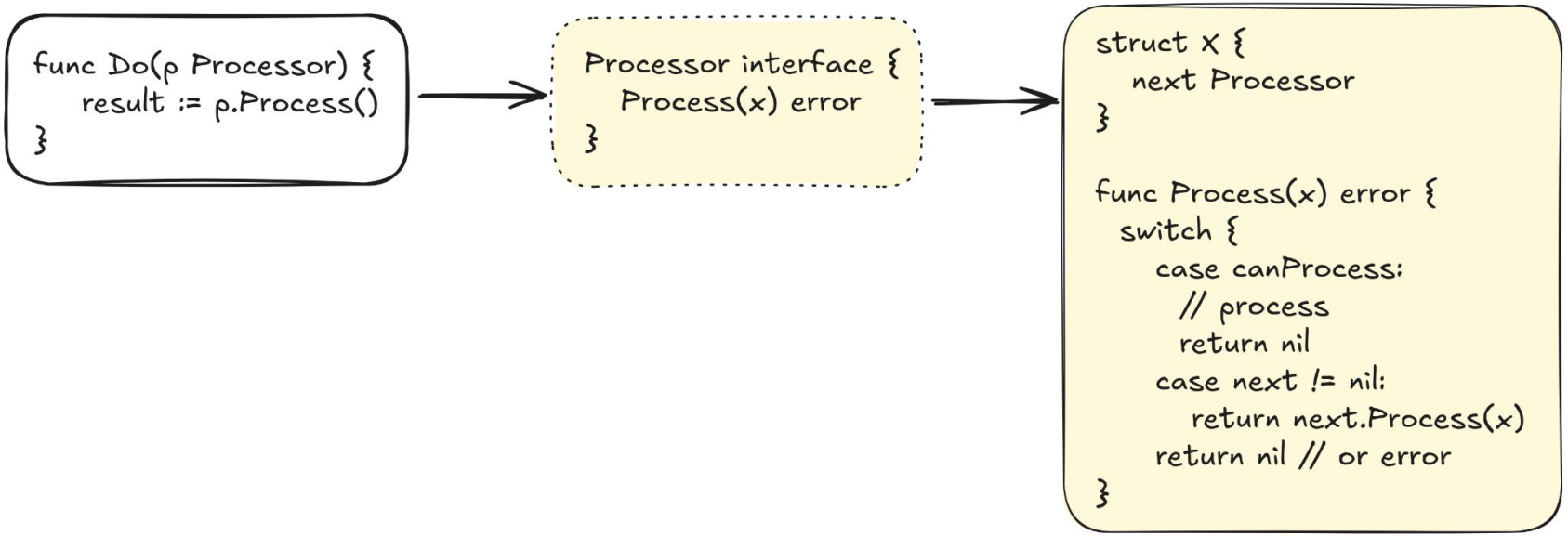


# Цепочка обязанностей



Цепочка обработчиков запроса

- отправитель не знает, кому придет запрос
- обработчик обрабатывает запрос сам или передает дальше

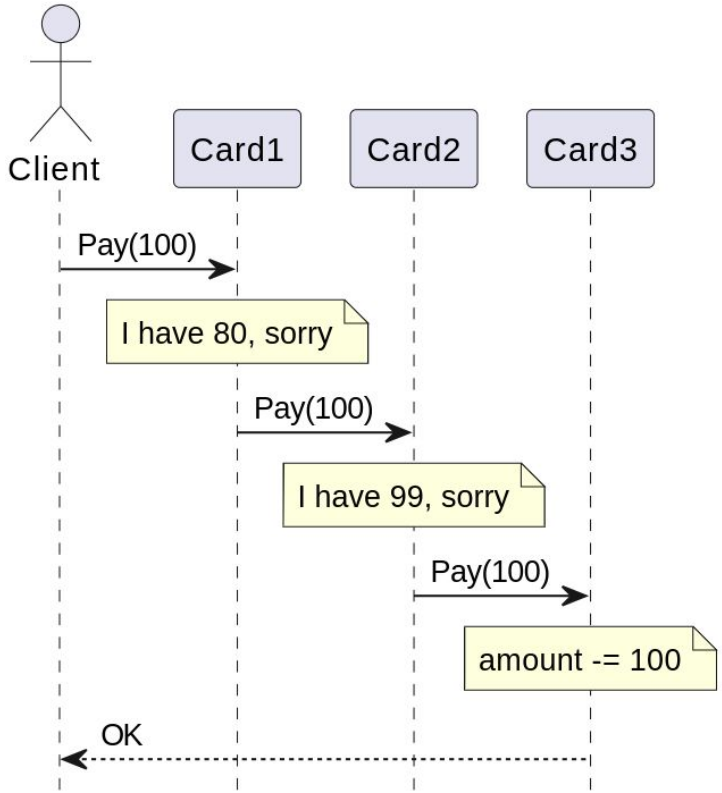


# Цепочка обязанностей



```
type Payer interface {
    func Pay(int) error
}


func (c *Card) Pay(x int) error {
    if c.amount < x {
        if c.next ≠ nil {
            return c.next.Pay(x)
        }
        return ErrGoToWork
    }
    c.amount -= x
    return nil
}
```



# Цепочка обязанностей

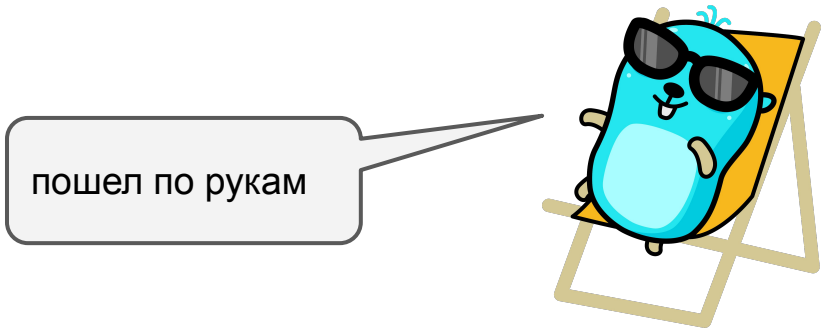


👍 Плюсы: гибкость, слабая связность

⛔ Минусы: запрос может не обработаться, код сложнее

🆚 Альтернативы: обычный цикл

🏁 Вердикт: может быть

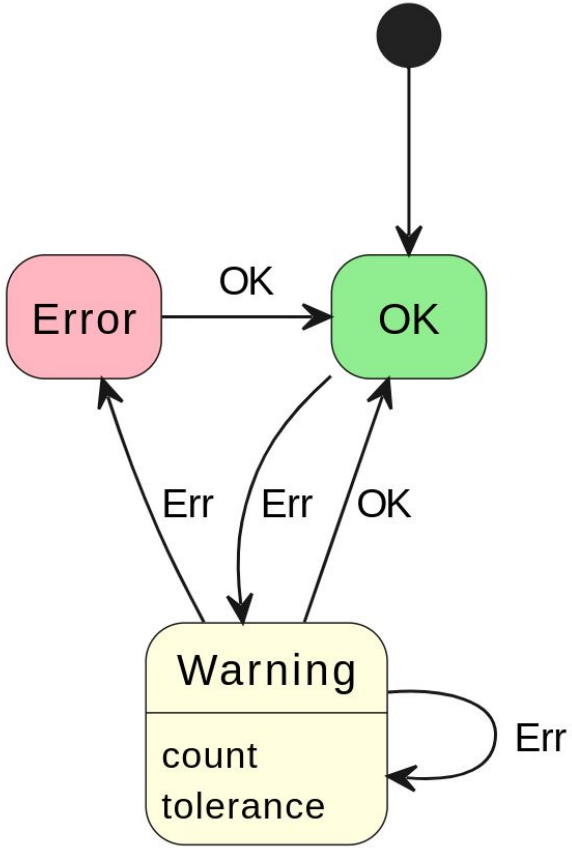


# Состояние



Стейт-машина

- меняет поведение объекта в зависимости от внутреннего состояния
- впечатление, что изменился класс объекта



# Состояние



```
Health struct {
    state State
    maxErr int
    currErr int
}

func (h *Health) HandleOK() {
        h.state.handleOK(h)
}

func (h *Health) HandleErr() {
        h.state.handleErr(h)
}
```

```
State interface {
    handleOK(*Health)
    handleErr(*Health)
    print(*Health) string
}
```



# Состояние



```
type ok struct{}

func (o ok) handleOK(h *Health) { }

func (o ok) handleErr(h *Health) {
   h.errCount++
   h.state = warn{ }
}
```

# Состояние



👍 Плюсы: гибкость, SRP, ветвления поделены между состояниями, проще логика переходов

⛔ Минусы: переусложнение при малом количестве состояний

🆚 Альтернативы: стратегия, таблица

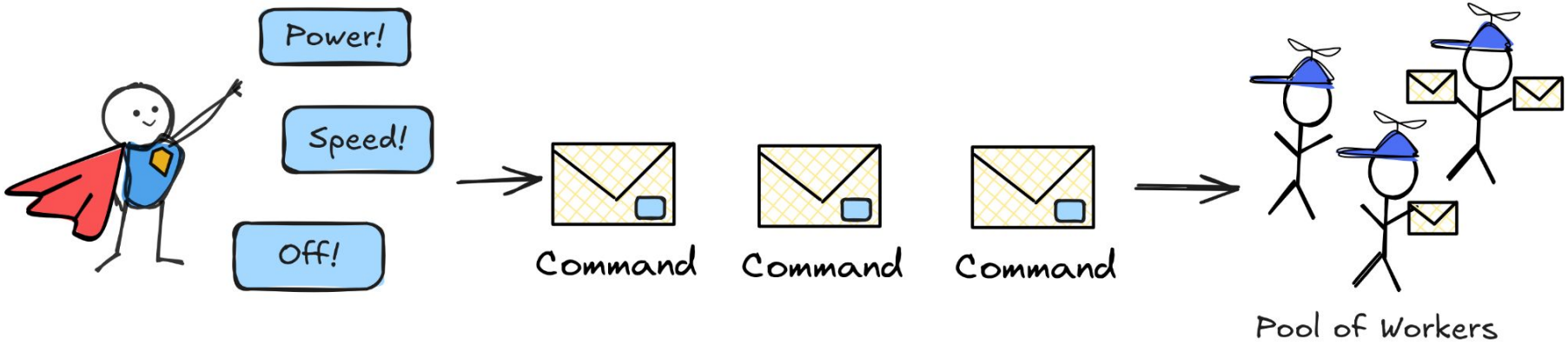
🏁 Вердикт: да!

# Команда



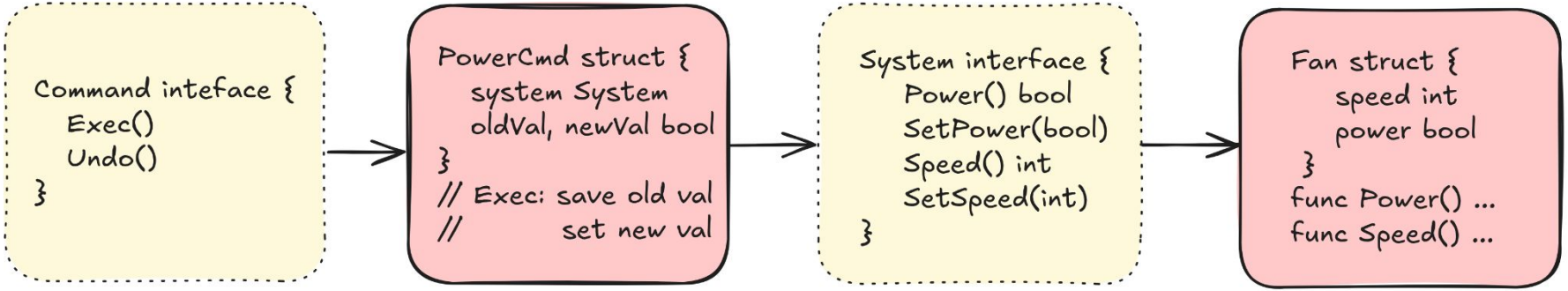
Запрос-объект

- отвязывает отправителя от исполнителя
- исполнитель не знает о конкретном действии
- можно сделать отмену команд



# Команда






```
func (p *PowerCmd) Execute() {
   p.oldVal = p.system.Power()
   c.system.SetPower(p.newValue)
}
```

# Команда



👍 Плюсы: отделение исполнителей от инициаторов, очереди команд, отложенный запуск, отмена, сортировка

⛔ Минусы: много кода

🆚 Альтернативы: список или канал пар функций-замыканий (exec & undo)

🏁 Вердикт: возможно

# Хранитель



Хранение и восстановление снимков состояний объекта

- состояние хранится вне объекта - SRP, гибкость
- данные все еще инкапсулируются



# История с отменой ..



```
type state struct {
   compression bool
   bandwidth   int
}

type System struct {
   current state
   history []state
}

func (s *System) SetCompression(v bool) { .. }
func (s *System) SetBandwidth(v int) { .. }
func (s *System) Compression() bool { .. }
func (s *System) Bandwidth() int { .. }
```

# История с отменой ..



```
func (s *System) Save() { .. }
func (s *System) Restore() { .. }
```

Как разделить основную логику и управление версиями, например,

сохранение на диск или по сети, от разных объектов?

И при этом скрыть реализацию этих объектов?

# Хранитель



```
Snap struct {
    data state
}

func (s Snap) data() data { .. }

func (s *System) Save() *Snap { .. }

func (s *System) Restore(m Memento) {
        s.state = m.state()
}
```

```
Memento interface {
    // только для своих
    state() state
}
```

package system

```
Backup struct {
    mementos []Memento
}

func (b *Backup) Save(s *System) {
    // save memento from s.Save()
}

func (b *Backup) Restore(s *System) {
    // take latest memento
    s.Restore(m)
}
```

package backup

# Хранитель



👍 Плюсы: SRP, инкапсуляция, хранение версий

⛔ Минусы: сложность

🆚 Альтернативы: внутренняя история версий, сериализация

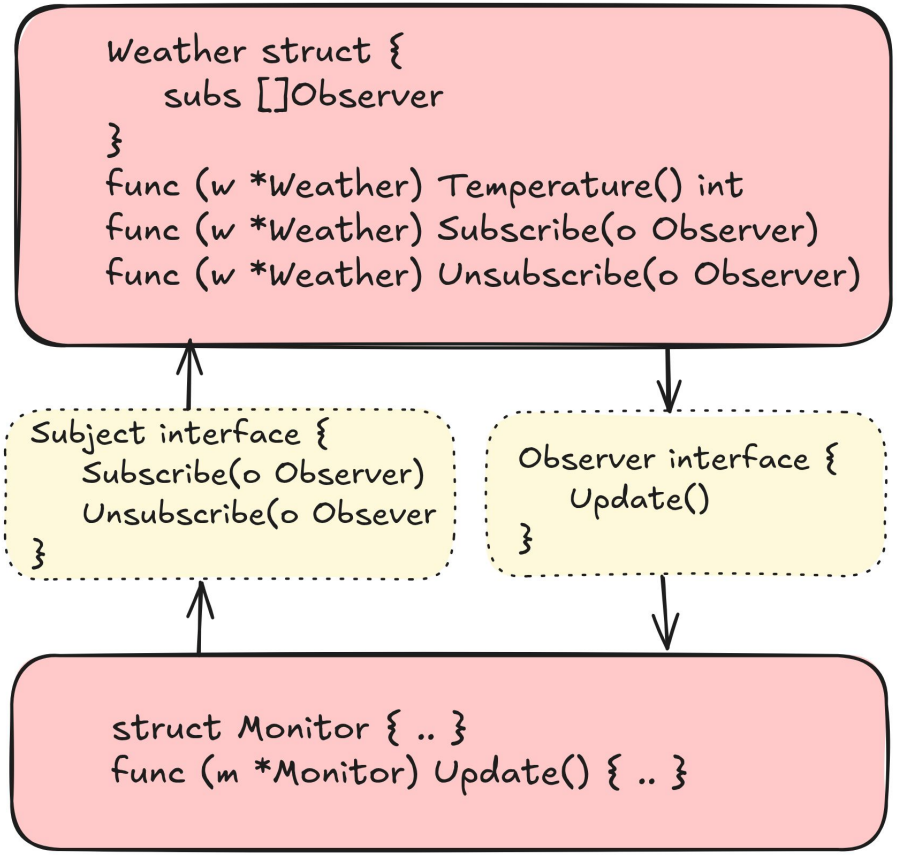
🏁 Вердикт: может быть!

# Наблюдатель



Механизм подписки на события в других объектах

- нет ненужных poll() / push()
- подписчики не знают издателей и наоборот



# “Зевака” - хочет знать все о всех



```
func (p *Person) Update(s Subject) {
    switch v := s.(type) {
    case *Clock:
        _ = v.Time()
    case *Weather:
        _ = v.Temperature()
    default:
        panic(o)
    }
}
```

# Наблюдатель



👍 Плюсы: обработка событий, простота

⛔ Минусы: много нюансов (отписки, конкурентность)

🆚 Альтернативы: polling, pushing, mediator

🏁 Вердикт: да!

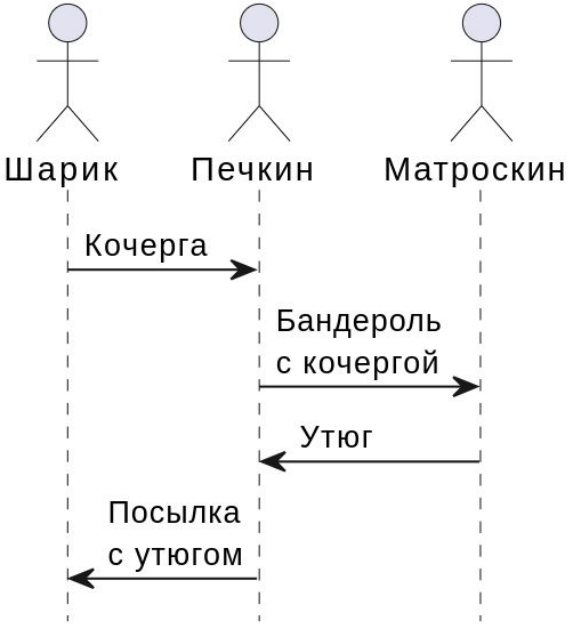
# Посредник

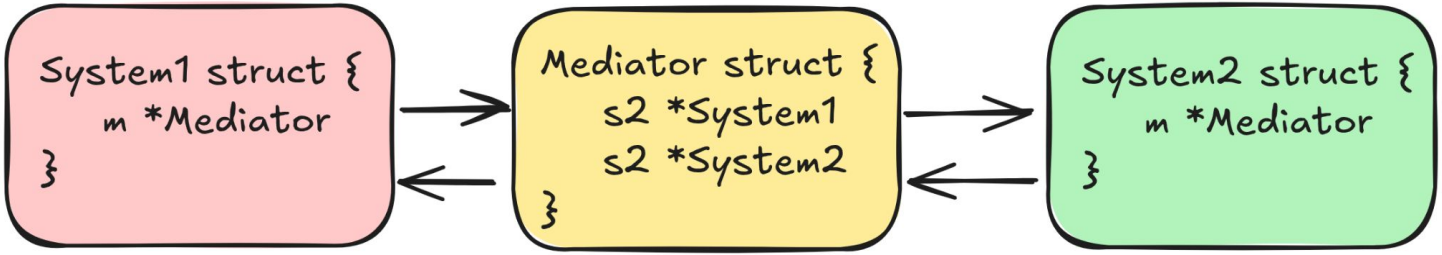


Обеспечивает взаимодействие множества объектов

- слабая связность
- централизация управления
- упрощение:

“все со всеми” → ”все с одним”



# Посредник



👍 Плюсы: слабая связность, SRP

⛔ Минусы: непонятно, что происходит, циклические ссылки и  конструкторы

🆚 Альтернативы: observer, bridge, facade, channels

🏁 Вердикт: если придется



# Итератор



Отделяет алгоритм обхода коллекции от самой коллекции

- инкапсуляция устройства коллекции
- разнообразие алгоритмов
- одновременные обходы
- единый интерфейс
- внешние и внутренние (pull vs push)

```
type Iterable[T any] interface {
        Iterator() Iterator[T]
}
```

```
type Iterator[T any] interface {
        Next() T
        HasNext() bool
}
```

# Итератор Go-way



```
type Iterable[T any] interface {
   Iterator() iter.Seq[T]
}

type List[T any] []T

func (l List[T]) Iterator() iter.Seq[T] {
   return func(yield func(T) bool) {
       for _, v := range l {
           if !yield(v) { return }
       }
   }
}
```

# Итератор



👍 Плюсы: понятность, можно дженерики, свои алгоритмы

⛔ Минусы: не работает range

🆚 Альтернативы: composite, golang iterators, просто map / slice / chan

🏁 Вердикт: скорее нет



# Посетитель



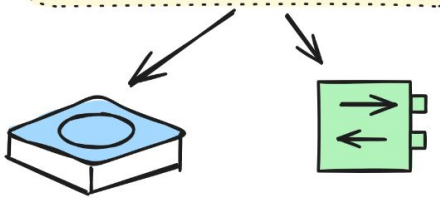
Нужно применить одну и ту же операцию к объектам разных типов, но без дублирования кода

- Хочется отделить алгоритм обработки от структуры данных
- можно добавлять новые операции
- типы объектов не меняются
- двойная диспетчеризация

# "Визитер"



```
type Element interface {
    Accept(Visitor)
}
```



```
type Visitor interface {
    VisitNIC(*NIC)
    VisitDisk(*Disk)
}
```

```
type PowerOnCommand bool

func (p PowerOnCommand) VisitDisk(d *Disk) {
    d.power = true
}

func (p PowerOnCommand) VisitNIC(n *NIC) {
    n.power = true
}
```

```
type HealthQueryCommand {
    health bool
}
// VisitDisk, VisitNIC
```

# "Посещаемые"



```
type NIC struct {
   health bool
   power  bool
}

func (n *NIC) Accept(v Visitor) { v.VisitNIC(n) }

type Disk struct {
   health bool
   power  bool
}

func (d *Disk) Accept(v Visitor) { v.VisitDisk(d) }
```

# "Посещение"



```
elements := []Element{&Disk{}, &NIC{}}

power := PowerOnCommand{}
for i := range elements {
    elements[i].Accept(power)
}

health := HealthCheckCmd{}
for i := range elements {
    elements[i].Accept(health)
}

print := PrintCommand{} ...
```

# Посетитель



👍 Плюсы: гибкость, новые операции - очень просто

⛔ Минусы: сложнее добавить новые элементы, открытость элементов

🆚 Альтернативы: composite, iterator

🏁 Вердикт: возможно



# Интерпретатор



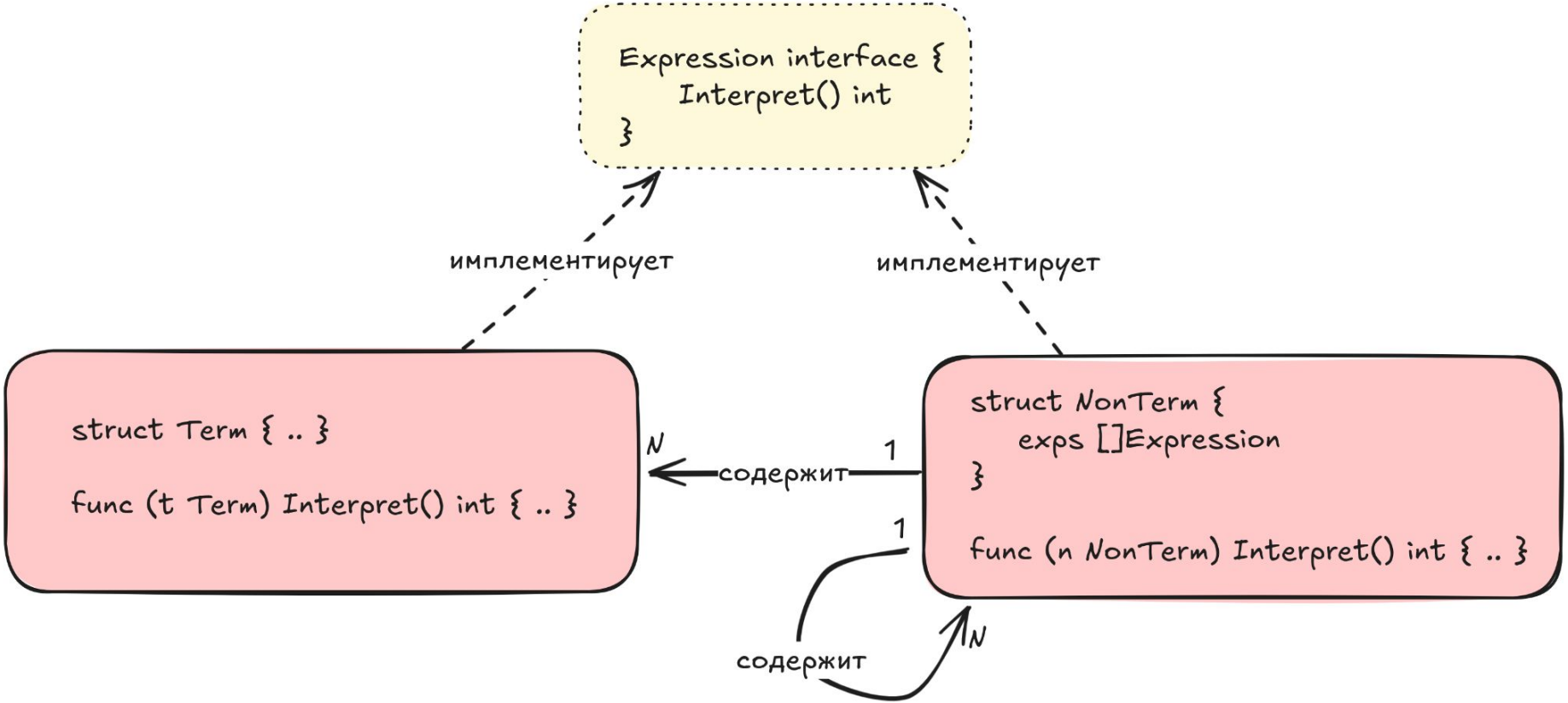
Определяет грамматику **простого** языка

- представляет выражения
- интерпретирует выражения
- SQL парсеры, конфиги, скрипты
- каждый класс - выражение, умеющее себя интерпретировать

В Go нет наследования, зато есть композиция интерфейсов

# Интерпретатор 1 + 2 + 3





# Интерпретатор 1 + 2 + 3



```
type Number int

func (n Number) Interpret() int {
   return int(n)
}

type Add struct {
   Left, Right Expression
}

func (a Add) Interpret() int {
   return a.Left.Interpret() + a.Right.Interpret()
}
```

# Интерпретатор



👍 Плюсы: простота изменения и расширения грамматики

⛔ Минусы: расход памяти, медленный, только простые грамматики

🆚 Альтернативы: regexp, компиляторы, парсеры

🏁 Вердикт: в классическом понимании нет, но в композиции?



# Ставим запятую …



| | порождающие | структурные | поведение |
|---|---|---|---|
| классы | • фабричный метод | • адаптер | • интерпретатор<br>• шаблонный метод |
| объекты | • абстрактная фабрика<br>• одиночка<br>• прототип<br>• строитель | • адаптер<br>• декоратор<br>• заместитель<br>• компоновщик<br>• мост<br>• приспособленец<br>• фасад | • итератор<br>• команда<br>• наблюдатель<br>• посетитель<br>• посредник<br>• состояние<br>• стратегия<br>• хранитель<br>• цепочка обязанностей |



удаляем

подумаем еще